LECTURE NOTES · OPEN MODELS, 2026

Post-Training, Visually

How a base model becomes an assistant: supervised fine-tuning, preference optimization, reinforcement learning, reasoning, long context, safety and tool use, as practised on open-weight models.



A base model is a very well-read person who has never been asked a question. Post-training does not make the person more learned; it teaches them to notice they are being asked something, to answer in a form the asker can use, to prefer good answers to bad ones, to check their work, to say no to some requests, and to pick up a tool when one is needed.

Every stage in the staircase is one of those lessons.

Terms

pretraining Training on next-token prediction over trillions of tokens of text and code. It produces the base model and consumes almost all of the compute.

mid-training A late phase of pretraining with a changed data mix (higher quality, more code and math, long documents) and usually the context extension. The term is informal; some reports fold it into pretraining and others into post-training.

supervised fine-tuning (SFT) Continuing to train the base model on prompt–response demonstrations with the next-token loss on the response. Also called instruction tuning when the demonstrations are instructions and answers.

preference optimization Training from comparisons between responses rather than from a single correct response. RLHF with a reward model and DPO are the two main forms.

reinforcement learning (RL) Training in which the model samples responses, receives a scalar reward for each, and is updated to make high-reward responses more likely. The data are produced by the model during training rather than collected beforehand.

demonstration A prompt paired with a response that the model should learn to produce. The data item of SFT.

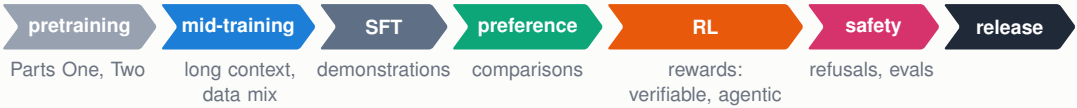
comparison A prompt with two (or more) responses and a judgment of which is better. The data item of preference optimization.

reward A scalar score assigned to a response, by a learned reward model, a program that checks the answer, or a judge model. The data item of RL.

assistant The post-trained model in its deployed role: it takes a conversation with system, user and assistant turns and produces the next assistant turn.

From base model to assistant

Pretraining produces a text predictor. Post-training turns it into something that answers, reasons, refuses, and acts.



prompt

Explain why the sky is blue.

base model continues the text

Explain why grass is green. Explain why… (Chapter 3 review questions, p. 41)

prompt

Explain why the sky is blue.

post-trained model answers

Sunlight is scattered by air molecules, and shorter (bluer) wavelengths scatter more (Rayleigh scattering), so…

Behavior, mostly not knowledge. The base model already "knows" the answer; post-training changes which continuation it produces, and in what form.

Three signals. Demonstrations (SFT), comparisons (preference optimization) and rewards (RL), in that order of cost and reach (next slide).

What changed by 2026. RL moved from last step to center, and several specialist RL runs are merged into one model by on-policy distillation (slide 28).

Three kinds of signal, three objectives

Every post-training method is one of these, or a combination. The data item decides the loss.

Demonstrations

Data item: a prompt and a good response, written by a person or a stronger model

Objective: maximize $\log \pi(y \mid x)$ on the response tokens, as in pretraining

Teaches: format, style, what a good answer looks like; cheap to get from a stronger model

Comparisons

Data item: a prompt and two responses, labelled with which is better

Objective: raise $\pi(y_w)$ relative to $\pi(y_l)$: a Bradley–Terry reward model, or DPO directly

Teaches: taste: tone, helpfulness, harmlessness; needs no correct answer, only a preference

Rewards

Data item: a prompt; the model samples responses; a verifier or reward model scores each

Objective: policy gradient: raise the probability of responses that scored above average

Teaches: search for behaviors in no dataset; the one signal that scales with compute

The order is also the history: SFT (FLAN, InstructGPT 2021–22), then comparisons (RLHF 2022, DPO 2023), then rewards from verifiers at scale (DeepSeek-R1 2025). Each later signal is more expensive per example and reaches behaviors the earlier ones cannot.

SFT I: the objective is pretraining loss on the answer only

A conversation is rendered through a chat template; the loss is taken on assistant tokens and masked everywhere else.



$$\mathcal{L}_{\text{SFT}} = -\sum_{t \in \text{assistant}} \log \pi_\theta(y_t \mid x,\, y_{<t})$$

Same loss as pretraining, restricted to the tokens the model should learn to produce. Multi-turn data masks every user turn and every tool output; only the assistant's own text is trained.

The template is part of the model. Special tokens for roles and turn ends are added to the vocabulary; the model learns to stop at <|end|>.

Practice

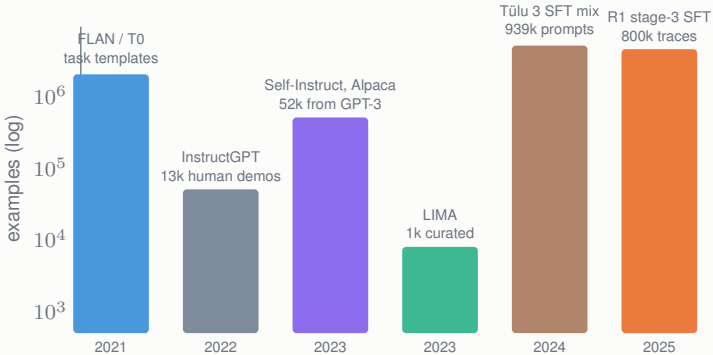
Hyperparameters. Learning rate 10^{-5} to 2×10^{-5} for full fine-tuning, 1–3 epochs, sequences packed to the context length; small, and easy to overshoot.

Full fine-tuning or LoRA. LoRA (Hu et al. 2021) trains low-rank adapters on frozen weights, QLoRA on 4-bit weights: a 70B model on one GPU, at some cost in quality on large data.

What SFT cannot do. It imitates: it cannot tell a wrong demonstration from a right one, and it drifts toward the demonstrators' style. Forgetting grows with epochs.

SFT II: where the demonstrations come from

Five years of instruction data: task templates, human demos, synthetic sets, a curated thousand, a million-prompt mix, R



sizes from the papers; FLAN templated 62 datasets capped at 30k examples each

academic tasks · **human-written** · **synthetic from a stronger model** · **curated** · **mixture** · **rejection-sampled from RL**

Three lessons

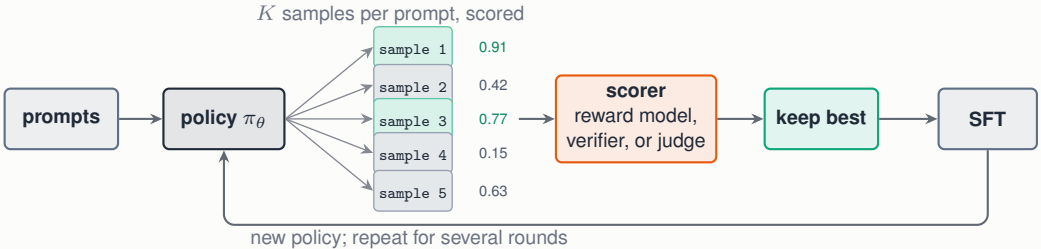
Quality. LIMA: 1,000 careful examples teach the format.

Coverage. Tülu 3: a broad mix with per-skill sources beats any single set.

Distillation. Most open SFT data today is generated by a stronger model and filtered, not written by people.

SFT III: rejection sampling, the bridge to RL

Sample many answers from the model itself, keep the ones a reward model or verifier likes, fine-tune on those, repeat.



Why it works. The model's own best samples are on-distribution (no exposure bias) and already in its style; the scorer supplies the judgment SFT lacks. It is RL with a very simple update.

Names for the same loop: expert iteration; STaR (Zelikman et al. 2022, keep rationales that reach the right answer); Llama 2 and 3's rejection sampling; DeepSeek-R1's stage-3 SFT on 800k filtered traces.

Where it sits in a 2026 recipe

Llama 3 ran six rounds of rejection sampling → SFT → DPO, with a reward model picking the samples, and no online RL. DeepSeek-R1 uses the loop after reasoning RL to turn a specialist's behavior into SFT data for the general model; it is also how reasoning traces for distillation are made (slide 17).

Terms

objective The quantity the optimizer maximizes or minimizes during training; the loss is the negative of it. Each signal has its own objective, written on the card.

policy The model viewed as a probability distribution over responses given a prompt, $\pi_\theta(y \mid x)$. The word comes from reinforcement learning, where the model's choice of next token is an action.

policy gradient The family of RL algorithms that update the policy's parameters in the direction that raises the log-probability of actions in proportion to how much better than expected they turned out. REINFORCE, PPO and GRPO are all policy-gradient methods.

verifier A program that decides whether a response is correct: an answer checker, a set of unit tests, a constraint checker. It supplies a reward without a learned model.

reward model (RM) A neural network trained on comparisons to output a scalar score for a response. It replaces the human judge during RL.

synthetic data Training examples generated by a model rather than written by people. Most open SFT and preference data today is synthetic, produced by a stronger model and filtered.

Bradley–Terry model A model of pairwise comparisons in which the probability that a beats b is a

RLHF: the original three steps

InstructGPT (2022): demonstrations, then comparisons, then optimization against the learned preference.

1 SFT

Labelers write the ideal response to prompts from the API; the base model is fine-tuned on them.

13k prompts

2 Reward model

Labelers rank several model outputs per prompt; a copy of the model with a scalar head learns to predict the ranking.

33k prompts

3 PPO against the RM

The policy samples responses, the reward model scores them, PPO raises the probability of high scorers, with a KL penalty to the SFT model.

31k prompts

Result. Labelers preferred the 1.3B InstructGPT's outputs to the 175B GPT-3's: a hundredfold size gap closed by a few tens of thousands of human judgments.

Why comparisons. Judging which of two answers is better is far easier and more consistent than writing the best one; the reward model turns cheap judgments into a dense signal.

Lineage

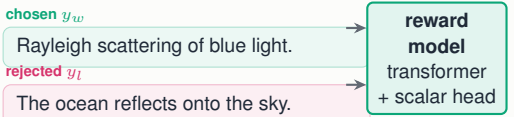
Christiano et al. 2017 (preferences for game agents) → Stiennon et al. 2020 (summarization) → Ouyang et al. 2022 (InstructGPT) → Bai et al. 2022 (HH-RLHF). Same algorithm; what grew was scale and prompt breadth.

Intuition

> Three ways to teach someone to cook. Show them a finished dish and the recipe (a demonstration); they will reproduce it, and only it. Taste two of their attempts and say which was better (a comparison); they can now improve what they already cook. Let them cook, taste the result, and score it (a reward); now they can invent. The third is the only one that can produce a dish you could not have made yourself, and the only one where a broken palate on the judge's part is catastrophic.

Reward models, and why you cannot optimize them forever

A Bradley–Terry model of preferences gives a scalar reward. Push the policy too far against it and true quality falls.



$$P(y_w \succ y_l) = \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)$$

$$\mathcal{L}_{\text{RM}} = -\log \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)$$

Bradley–Terry. The chance that y_w wins is a logistic function of the reward gap; rewards are defined up to a constant per prompt. Data: tens of thousands to millions of pairs, from people or from a stronger model (RLAIF).

Variants. Pairwise classifiers that read both answers at once; generative reward models that write a verdict; rubric judges.



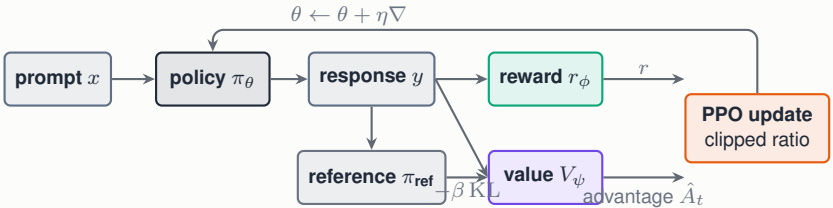
Overoptimization (Gao, Schulman, Hilton 2022): with a gold RM as ground truth, $R_{\text{RL}}(d) = d(\alpha - \beta \log d)$. The proxy keeps rising; the gold peaks, then falls. Bigger RMs and more RM data push the peak out. Goodhart's law, measured.

Intuition

A student who studies only the past exam papers will do better on the next exam, up to a point, and then start doing worse, because they have begun to learn the examiner's habits rather than the subject. The reward model is the past papers. The KL penalty is a rule that says: study, but do not become unrecognizable to the teacher who wrote the papers.

PPO for language models

Policy gradient with a clipped ratio, a KL leash to the reference model, and a value model: four networks in memory.



$$\mathcal{L}_{\text{PPO}} = \mathbb{E}_t\Big[\min\big(\rho_t\hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big], \qquad \rho_t = \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t \mid x, y_{<t})}, \qquad r_t = r_\phi(x,y)\mathbb{1}[t{=}T] - \beta\,\mathrm{KL}_t$$

Four models. Policy and value train; reference and reward are frozen. At 70B that is four 70B networks plus optimizer state, which is why GRPO (slide 12) and DPO (slide 10) exist.

The KL leash $\mathrm{KL}_t = \log \pi_\theta(y_t \mid \cdot) - \log \pi_{\text{ref}}(y_t \mid \cdot)$ keeps the policy near the SFT model, so the reward model is queried only where it was trained (slide 8).

Knobs that matter

Clip $\epsilon = 0.2$; KL coefficient $\beta \sim 0.01$–0.1, often adapted to a target KL; GAE advantages; one reward per sequence at the last token; a few optimizer steps per batch before resampling. Failure modes: reward hacking, length growth, value-model lag.

DPO: skip the reward model

The KL-regularized objective has a closed-form optimum; put into Bradley–Terry, it gives a loss on the policy alone.

1. Objective: $\max_\pi \ \mathbb{E}_{y\sim\pi}[r(x,y)] - \beta\,\mathrm{KL}(\pi \,\|\, \pi_{\text{ref}})$
2. Optimum: $\pi^*(y \mid x) = \dfrac{1}{Z(x)}\,\pi_{\text{ref}}(y \mid x)\, e^{r(x,y)/\beta}$
3. Invert: $r(x,y) = \beta \log \dfrac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x)$
4. Bradley–Terry: $Z(x)$ cancels in $r(x,y_w) - r(x,y_l)$

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}\Big[\log \sigma\Big(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\Big)\Big]$$

Two models, one classification loss, no sampling during training. The implicit reward is $\beta \log(\pi_\theta/\pi_{\text{ref}})$; $\beta \approx 0.1$ plays the role of the KL coefficient.

RLHF

SFT → RM → PPO — 4 models, samples

DPO

SFT → DPO — 2 models, no samples; off-policy

Variants and the catch

IPO (bounded objective); **KTO** (unpaired labels); **SimPO** (no reference, length-normalized); **ORPO** (SFT and preference in one stage).

The catch. DPO learns from pairs someone else sampled; as the policy moves they go off-policy and the implicit reward is trusted where it was never checked. Iterative DPO resamples each round; by 2026 most frontier recipes use RL instead.

Choosing a preference method

What each needs, what it costs, and who used what. The trend from 2024 to 2026 is toward on-policy methods.

Method	Reward model	Value model	On-policy	Nets	Signal	Typical use
DPO (2023)	no	no	no	2	pairs	chat preferences; Llama 3, Tülu 3, OLMo 3
SimPO, ORPO	no	no	no	1	pairs	cheap preference stage, small models
PPO (2017/22)	yes	yes	yes	4	scalar per response	InstructGPT, Llama 2; now rare in open recipes
RLOO, REINFORCE	RM or verifier	no	yes	2–3	scalar, leave-one-out baseline	simple; competitive with PPO
GRPO (2024)	RM or verifier	no	yes	2–3	scalar, group-normalized	reasoning RL: R1, Qwen3, most RLVR since

Rule of thumb. DPO when you have pairs and no verifier (style, helpfulness, safety). GRPO or RLOO when you can score a response (math, code, instruction constraints) or have a strong reward model and the budget to sample.

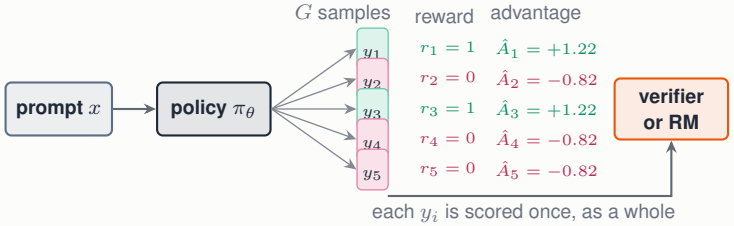
What the open recipes did. Llama 3 (2024): rejection sampling, SFT, DPO over six rounds, no online RL. Tülu 3 (2024): SFT → DPO → RLVR. OLMo 3 (2025): the same three stages for each of its Think, Instruct and RL-Zero variants.

2026: DPO is receding

Frontier open recipes of 2026 (DeepSeek-V4, GLM-5, Kimi K2.5, Nemotron 3) center on RL with verifiable and rubric rewards, and several report no DPO stage; the preference signal enters RL as a reward model or judge. This is from the labs' write-ups and commentary, not controlled ablations; where ablations exist (Nemotron 3), a preference stage still helped.

GRPO: group-relative advantages, no value model

Sample a group of answers per prompt, score them, and use each answer's standing in its group as the advantage.



$$\hat{A}_i = \frac{r_i - \mathrm{mean}(r_1..r_G)}{\mathrm{std}(r_1..r_G)}$$

here: mean 0.4, std 0.49

$$\mathcal{J}_{\text{GRPO}} = \mathbb{E}\Big[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|y_i|}\sum_t \min\big(\rho_{i,t}\hat{A}_i,\ \mathrm{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon)\hat{A}_i\big) - \beta\,\mathbb{D}_{\mathrm{KL}}\big[\pi_\theta \,\|\, \pi_{\text{ref}}\big]\Big]$$

What is gone. No value model: the group mean is the baseline, so the critic's job is done by the $G-1$ siblings. Two or three networks instead of four; the compute goes into sampling.

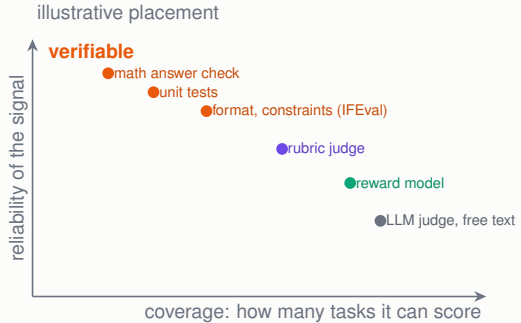
What is kept. PPO's clipped ratio and a KL term, here added to the loss with an unbiased estimator rather than folded into the reward.

Origin and fit

Introduced in DeepSeekMath (Shao et al. 2024); the engine of DeepSeek-R1 and of most 2025–26 reasoning RL. It suits sparse, sequence-level rewards (right or wrong), since the advantage is one number per response; it is a poor fit for dense per-token rewards. G is 8–64; prompts where all G agree carry no gradient.

RL with verifiable rewards (RLVR)

When a program can check correctness, the reward model disappears and there is far less to hack.



Named by Tülu 3 (2024), which ran it on GSM8K, MATH and IFEval after DPO; DeepSeek-R1-Zero used only accuracy and format rewards. The signal is cheap, exact and un-gameable at the answer level, so the policy can be pushed much further than against a learned reward model.

verifiable reward, math

1 if the normalized boxed answer equals the gold answer, else 0; often plus a format reward for using the `<answer>` tags

verifiable reward, code

run the hidden unit tests in a sandbox; reward = fraction passed, or 1 only if all pass

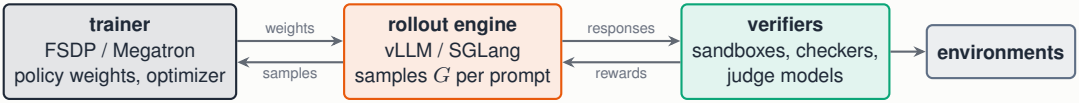
verifiable reward, instructions

check each constraint by program: “exactly three bullets”, “no commas”, “ends with a question”

Still hackable at the edges. Tests can be edited, answers guessed in several formats, constraints met with nonsense. Mitigations: sandboxes, canonical answer extraction, a judge for the rest. **Rubric rewards** (Kimi K2’s self-critique; generative reward models) extend coverage to non-verifiable domains at some cost in reliability.

The RL loop at scale, and what breaks

Most of the compute is generation; the trainer and the inference engine run side by side.



asynchronous: generation runs a step or two ahead of the trainer; stale samples are reweighted or filtered

Train/inference mismatch

Engine and trainer kernels give slightly different token probabilities, so ρ drifts off-policy. Fixes: recompute log-probs in the trainer, importance weights, or mask the worst tokens (IcePop).

Entropy collapse and length

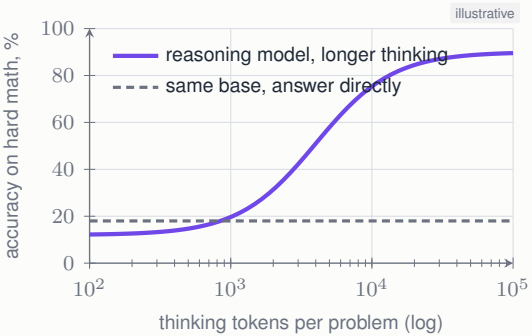
The policy narrows onto a few styles and stops exploring; answers grow long. DAPO: clip-higher, drop prompts where all G agree, token-level loss, soft penalties for overlong answers.

Biased estimators

Dr. GRPO: the $1/|y_i|$ normalization penalizes long wrong answers less, so length grows; the group-std normalization over-weights easy and hard prompts. Removing both gives an unbiased gradient.

Thinking tokens: spend compute at inference

Models that write out their reasoning improve with more thinking tokens; RL taught them to use them.



The test-time scaling curve: accuracy rises roughly log-linearly in tokens spent, up to the problem's ceiling. Each model has its own curve.

How we got here

Chain-of-thought prompting (Wei et al. 2022): ask for steps, get better answers; the capability was latent in large base models.

Bootstrapping (STaR 2022): keep the rationales that reach the right answer, fine-tune on them, repeat.

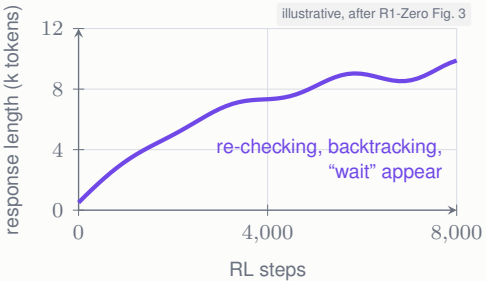
Process supervision (Lightman et al. 2023): reward each step; PRM800K; better than outcome-only on MATH.

Test-time compute (Snell et al. 2024): spent well, inference compute can beat a 14× larger model on some problems.

o1 (OpenAI, September 2024) and **DeepSeek-R1** (January 2025): RL on verifiable rewards teaches the model to think long on its own; R1 published the recipe.

DeepSeek-R1: RL makes the model think longer

R1-Zero: a base model, GRPO and two rule rewards; the reasoning behaviors appeared on their own.



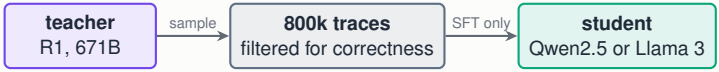
R1-Zero. DeepSeek-V3-Base, GRPO, reward = accuracy (rule check) + format (think tags). Response length and AIME accuracy rose together over training; self-verification and reflection emerged, the paper’s “aha moment”. Flaws: language mixing, poor readability.

1. **Cold-start SFT**
   thousands of long, readable chain-of-thought examples, to fix the format before RL
2. **Reasoning RL**
   GRPO on math, code and logic with rule rewards, plus a language-consistency reward
3. **Rejection-sampling SFT**
   about 800k samples: 600k correct reasoning traces plus 200k general writing and QA
4. **RL for all scenarios**
   rule rewards on reasoning prompts; reward models for helpfulness and harmlessness on the rest

Four stages, two SFT and two RL, alternating: RL discovers, SFT consolidates and broadens, RL polishes. The same shape recurs in Qwen3 and GLM-5.

Distillation, thinking budgets, and one model for both modes

Reasoning transfers by SFT on traces; a model can be taught to think only when asked, and to stop when told.



Distillation beats RL for small models. R1's distilled Qwen and Llama students, trained by SFT alone on the teacher's traces, outperformed the same sizes trained by RL from scratch: the search is expensive, the result is cheap to copy. **s1** (2025): 1,000 curated traces suffice to turn a 32B base into a reasoner.



Budget forcing (s1): suppress the end-of-thinking token and append "Wait" to make the model think longer; emit it early to make it think less. Thinking length becomes a dial.

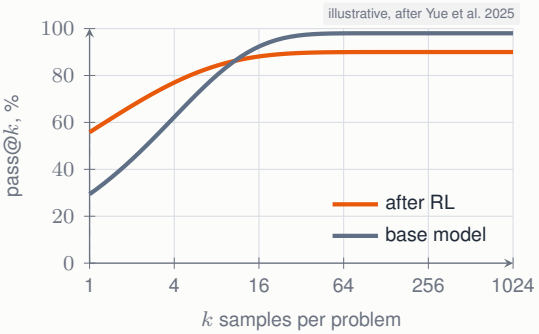
One model, two modes (Qwen3)

Stage 1: long-CoT cold start. Stage 2: reasoning RL. Stage 3: *thinking-mode fusion*, SFT on mixed thinking and non-thinking data with `/think` and `/no_think` flags, so one model serves both. Stage 4: general RL on many tasks.

A *thinking budget* caps the thinking tokens and forces the answer; quality rises smoothly with the budget. Most 2025–26 open models ship this way.

What reasoning RL does, and does not, add

Pass@1 rises a lot; pass@k at large k barely moves: RL mostly sharpens what the base model had.



At $k = 1$ the RL model is far ahead; by k in the hundreds the base model catches up. The right answers were already in the base distribution; RL moved probability onto them.

Reading it

Sharpening, not discovery (Yue et al. 2025): with current recipes, RL concentrates the policy on solutions the base model could already sample; distillation from a stronger teacher does add new ones. Whether longer RL on harder problems changes this is open.

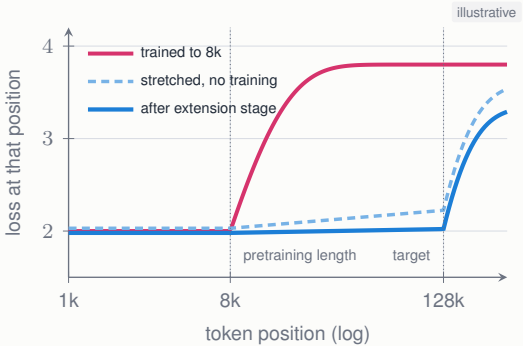
Overthinking. Length is rewarded indirectly (more chances to recover), so models think long on easy questions; length penalties and budgets address it.

Entropy collapse. Pass@1 rises as diversity falls; too much and the model cannot explore the next task. Clip-higher and entropy bonuses keep it open.

Evaluation. Report pass@1 averaged over many samples (AIME has 30 questions), never a single run.

Context: why a long window is its own stage

Models are pretrained at 4–8k tokens for cost; the window is stretched late, with a small fraction of the data.



Past the trained length the RoPE angles are ones the model never saw: attention spreads out and loss climbs within a few thousand tokens. Stretching the positions fixes most of it; a short training stage fixes the rest.

Why not pretrain long from the start?

Attention cost per layer grows with the square of the sequence length, and documents longer than 8k tokens are rare, so pretraining runs at 4–8k and the window is stretched at the end.

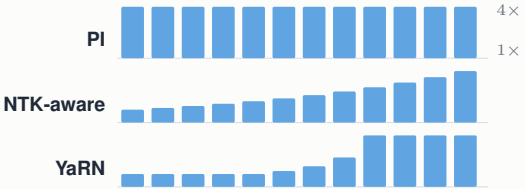
Three things change. The *position encoding* (RoPE frequencies are rescaled), the *data* (long documents up-sampled, plus synthetic long tasks), and the *systems* (sequence parallelism, FlashAttention, chunked prefill).

Where it sits. Usually between pretraining and SFT, as the last pretraining stage or “mid-training”: Llama 3 ran it on the 405B base before any SFT. The later stages must then keep the ability: the SFT mix needs long examples.

Stretching RoPE: interpolation, base change, YaRN

Every method rescales the rotation frequencies so that unseen positions map onto seen angles.

RoPE rotates query–key pair i at position m by the angle $m\,\theta_i$, with $\theta_i = b^{-2i/d}$: pair 0 turns once every few tokens, the last pair once in $\sim b$ tokens. Extending $L \to sL$ means choosing a new θ_i' for each pair.



Pair i runs from high frequency (left, short wavelength) to low frequency (right); bar height is how much positions are stretched for that pair, $s = 4$. PI stretches every pair. NTK-aware raises the base, so low frequencies stretch more. YaRN leaves the high-frequency pairs alone (they encode local order), interpolates the low ones, and cools the attention logits by $1/\sqrt{t}$ with $\sqrt{1/t} = 0.1 \ln s + 1$.

Method	What changes; training needed	Reach
PI (Chen et al. 2023)	positions $m \to m/s$ on all pairs; $\sim$1000 fine-tuning steps	2k→32k
NTK-aware (bloc97 2023)	base $b \to b\,s^{d/(d-2)}$; no training	small s
YaRN (Peng et al. 2023)	by-parts interpolation + attention temperature; $<$0.1% of the pretraining data	128k
ABF (Xiong et al. 2023)	base $10^4 \to 5 \cdot 10^5$, continue pretraining	32k
LongRoPE (Ding et al. 2024)	per-pair factors by evolutionary search; fine-tune at 256k	2048k

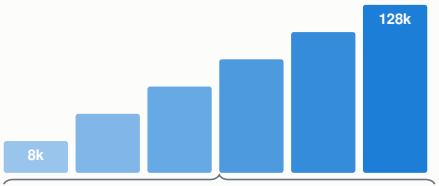
What open models do now

Raise the base early (Llama 3: $5 \cdot 10^5$; Qwen2.5 and Qwen3: 10^6), train the long stage at 32k–128k, and apply YaRN factors at inference for a further $4\times$ (Qwen3). DeepSeek-V3 ran two YaRN phases, 4k→32k→128k.

The long-context protocol: stages, data, and checks

Extend in steps, gate each step on recovery of short-context scores and a perfect needle test.

six stages, ≈800B tokens in total (Llama 3, 405B); intermediate lengths are not published



gate between stages: short-context evals back to their pre-stage values, needle-in-a-haystack solved perfectly

Data. Long documents up-sampled, plus synthetic tasks over long inputs. The SFT mix then needs long examples too (Llama 3: 0.1% synthetic long-context SFT data); DPO on short data alone did not hurt.

illustrative, after Liu et al. 2023



Lost in the middle. Facts at the start or end of the window are used; those in the middle are often missed. Position-aware evals, not just perplexity.

Checks

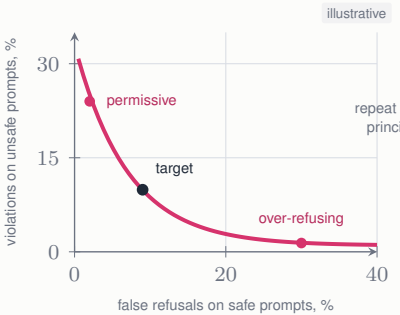
NIAH (Kamradt 2023): hide a sentence, ask for it; necessary, not sufficient.

RULER (Hsieh et al. 2024): 13 tasks in 4 categories (retrieval, multi-hop tracing, aggregation, QA); most models fall short of their claimed length.

Run every eval at every length, and re-run the short-context suite: the usual regression is there.

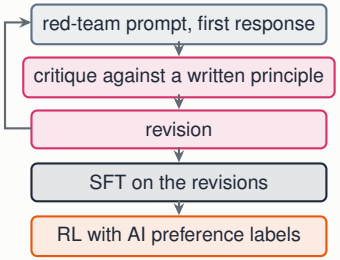
Safety alignment: targets and data

Helpful, honest, harmless. The stage tunes one trade-off and needs data on both sides of the line.



HHH (Askell et al. 2021). Safety training moves the model along this curve; Llama 2 and 3 report both rates and tune the data mix until both are acceptable. Borderline prompts (XSTest: 250 safe, 200 unsafe) measure the refusal side.

Constitutional AI (Bai et al. 2022)



harmlessness labels from the model (RLAIF); helpfulness labels from humans. Critique and revision text come from the model itself.

Where the data comes from

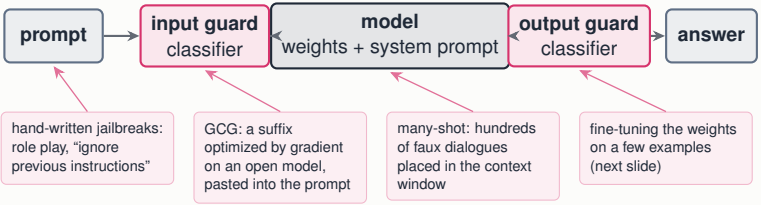
Human red teams write attacks; annotators choose the safer response. Llama 2 trained a separate *safety reward model* and used *context distillation*: answer under a safety preprompt, then train without it.

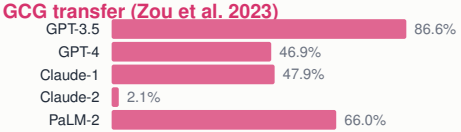
Model critiques (CAI) replace most human harmlessness labels.

Borderline prompts and safe completions, so refusal is not the only safe answer.

Measuring safety: red teams, jailbreaks and guards

The aligned weights are one layer. Optimized attacks transfer across models; classifiers and specs add layers.





attack success rate of suffixes optimized on Vicuna and Guanaco, transferred to closed models; Table 2, “ensemble” (any one suffix works), 388 behaviors. One open model’s gradients attack the others.

Benchmarks and tools

HarmBench (Mazeika et al. 2024): 510 behaviors, 18 attacks, 33 models, one scoring protocol.

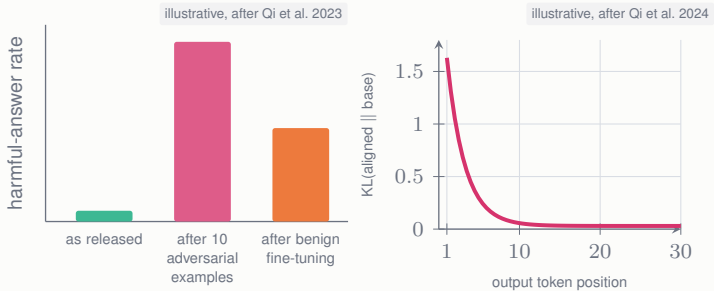
XSTest: over-refusal on safe prompts that look unsafe.

Llama Guard (Inan et al. 2023): an LLM classifier for inputs and outputs; version 3 covers 14 hazard categories.

Deliberative alignment (Guan et al. 2024): the model reads the written policy and reasons over it; SFT on policy-citing chains, then RL with a judge that holds the policy.

Open weights: fine-tuning undoes alignment

A few examples or a benign dataset erode safety; the alignment is only a few tokens deep.



Qi et al. 2023: fine-tuning GPT-3.5 Turbo on 10 adversarial examples, for under $0.20 through the API, removed its refusals; fine-tuning on a benign dataset (Alpaca) also degraded safety. **Qi et al. 2024**: the aligned and base models differ mainly in the first few output tokens, so a prefix is enough to steer around it. Fixes: train on recovery from harmful prefixes, and constrain fine-tuning to keep those first tokens.

What open releases do

Deepen the alignment with prefix-recovery data and a constrained fine-tuning objective.

Ship the other layers: Llama Guard and prompt classifiers released with the weights.

Remove hazardous knowledge: unlearning (WMDP, RMU) and tamper-resistant training (TAR) hold for hundreds of fine-tuning steps, not indefinitely.

Decide what to release: uplift evaluations before the weights go out. None of this is settled.

Tool use I: function calling is a format, taught by SFT

The model emits a structured call between special tokens; the runtime executes it and appends the result.

system (the chat template inserts the schemas) — masked

```
tools: [{"name": "get_weather",
"parameters": {"city": "string"}}]
```

user — masked

What is the weather in Chicago today?

assistant — loss

```
<tool_call>{"name": "get_weather",
"arguments": {"city":
"Chicago"}}</tool_call>
```

tool (the runtime ran it) — masked

```
{"forecast": "rain", "high_f": 54}
```

assistant — loss

Rain in Chicago today, with a high of 54 °F.

How it is taught

Schemas go into the system turn via the chat template; the call is emitted between special tokens (formats differ by family) and parsed by the runtime, which appends the result as a tool turn.

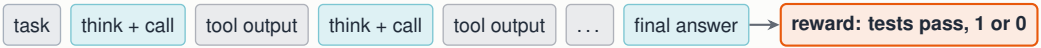
Data. Synthetic multi-tool dialogues checked by execution: APIGen (60k verified examples, 3,673 APIs), ToolBench (16,464 real APIs); plus negatives where no tool is needed. Loss on assistant tokens only.

Evaluation. BFCL (Patil et al. 2025): single, parallel and multi-turn calls; τ-bench (Yao et al. 2024): simulated users, pass^k.

Failures. Invented functions, wrong argument types, calling when not needed, ignoring the result.

Tool use II: agentic RL over many turns

The same policy gradient, but the trajectory is a long dialogue with an environment and one reward at the end.



Loss and advantage apply to the model's tokens only (teal); tool outputs are context and masked (grey). One scalar at the end is spread over every model turn; GRPO uses one advantage per trajectory, some recipes one per turn.

What changes against single-turn RLVR.

Trajectories run 10–100 turns and tool outputs are most of the tokens, so the context fills and rollouts are slow; generation must be asynchronous, since each turn blocks on a tool.

Environments must be sandboxed, resettable and fast: a container per rollout for code, a simulator for users or browsers.

Rewards are sparse and final; the policy has to learn recovery from tool errors, which only shows up in long trajectories.

Credit assignment is the open problem: which of forty calls mattered?

Where it is done

SWE-bench Verified: 500 human-validated GitHub issues (OpenAI, August 2024); reward = the hidden tests pass. The standard RL environment for coding agents.

Kimi K2 (2025): a synthetic pipeline with thousands of tools and simulated users; verifiable rewards where possible, a self-critique rubric reward elsewhere; agentic SFT data at scale before the RL.

Tooling. Agent loops in veRL and slime; NVIDIA's NeMo Gym environments; OpenRLHF. The rollout engine, not the trainer, is the bottleneck.

Open recipes, side by side

The shape is stable; the weight of each stage, and where the data comes from, moved between 2024 and 2026.

Model (date)	SFT	Preference	RL	Reasoning, context, safety, tools
Llama 3.1 405B (Meta, Jul 2024)	rejection-sampled and synthetic data, six rounds	DPO each round	none online	long context in pretraining; tools by SFT; safety data tuned on violation vs false-refusal rates
Tülu 3 (AI2, Nov 2024)	939k prompts, open mix	length-normalized DPO	RLVR: GSM8K, MATH, IFEval	fully open data, code and evals
DeepSeek-R1 (Jan 2025)	cold start, then rejection-sampling SFT (800k)	none; RMs in the final RL	GRPO, twice	R1-Zero from the base; distilled 1.5–70B students by SFT only
Qwen3 (Apr 2025)	long-CoT cold start; thinking-mode fusion	none reported	reasoning RL; general RL	thinking budget; small models by strong-to-weak distillation
Kimi K2 (Moonshot, Jul 2025)	with large synthetic agentic data	none reported	verifiable + self-critique rubric	1T/32B MoE; agentic data synthesis; MuonClip
OLMo 3 (AI2, Nov 2025)†	SFT	DPO	RLVR	Think, Instruct and RL-Zero variants
2026 releases†	SFT (DeepSeek-V4, GLM-5, MiMo-V2-Flash, Nemotron 3)	often none	staged RL, central	on-policy distillation from specialist teachers (MOPD); where ablated (Nemotron 3) a preference stage still helped

Reading it. SFT, then preference or RL, with reasoning RL from 2025: the sequence is stable; what moved is the weight of each stage (RL up, DPO down) and the data source (synthetic, checked by execution). †Release notes and commentary, not papers.

On-policy distillation: dense signal, student's samples

Between SFT and RL: the student samples, the teacher scores every token. The 2026 consolidation step.

	off-policy teacher's samples	**on-policy** student's samples
dense per token	**SFT on teacher traces** copies the teacher's tokens; off-distribution	**on-policy distillation** student samples; teacher scores every token
sparse per answer	**rejection-sampling SFT** keep samples a checker accepts; then SFT	**RLVR, GRPO** one reward per answer; the search costs

per-token reward on the student's sample y: $r_t = \log \pi_T(y_t \mid x, y_{<t})$

Maximizing it minimizes the reverse KL from student to teacher, token by token, on the student's own distribution: many bits per sample (RL gets one) and no exposure bias (SFT has it). The teacher must share the tokenizer, or be mapped onto it.

Who uses it

GKD (Agarwal et al. 2023): the origin.

Qwen3: the small models come from strong-to-weak distillation, off-policy then on-policy, at about a tenth of RL's GPU hours (1,800 vs 17,920 in the report's Table 21).

Thinking Machines (October 2025)†: reports order-of-magnitude compute savings against RL for reasoning.

Multi-teacher OPD (2026): specialist teachers (math, code, agents) into one student, the best teacher chosen per prompt; named by MiMo-V2-Flash, DeepSeek-V4 and Nemotron 3 Ultra†. The compute claims are the labs' own.

Evaluating each stage, and the regressions to watch

Every stage is checked by its own suite, and the previous suites are re-run, because the usual damage is elsewhere.

SFT	preference	RL	context	safety	tools
IFEval; MT-Bench; GSM8K, MATH; MMLU, to see what was forgotten	AlpacaEval 2 LC; Arena-Hard; answer length	AIME avg@32; LiveCodeBench; GPQA Diamond; pass@k	NIAH; RULER at every length; the short suite again	XSTest; HarmBench; violation vs false-refusal rates	BFCL; τ-bench; SWE-bench Verified

Regressions

Length creep (length-controlled metrics exist for a reason); over-refusal; knowledge loss on MMLU after RL; language mixing; format collapse; a drop on the previous stage's suite.

Reward hacking

Judges reward length, confidence and their own style; tests get edited; constraints are met with nonsense. Read samples, not only scores; hold out a judge the policy never saw.

Protocol

Same decoding settings at every stage (temperature, max tokens, system prompt); average over samples and seeds; held-out prompts and contamination checks; report variance.

A practical plan for a small team

One 7–8B base, open data, one node of GPUs: the whole pipeline in order, with indicative settings.

1. **Base and evals first**
   a 7–8B base with a known chat template; freeze the eval suite before training
2. **SFT**
   100k–1M examples (Tülu 3 mix, OpenThoughts3); full fine-tune on 8×80 GB, else LoRA; mask prompts
3. **Preference**
   DPO on 50–300k pairs (UltraFeedback, Tülu 3 preference mix); length-normalized if answers grow
4. **RLVR**
   GRPO on 10–50k checkable prompts (math, code, constraints); vLLM rollouts; start from the DPO model
5. **Safety, tools, release**
   5–10% of the SFT and DPO mix; a guard model at serving; re-run every suite, then loop

	SFT	DPO	GRPO
learning rate	$5 \cdot 10^{-6}$ to $2 \cdot 10^{-5}$	$5 \cdot 10^{-7}$ to 10^{-6}	10^{-6}
length	1–3 epochs	1 epoch	500–2000 steps
batch	64–256 seqs	32–128 pairs	128–512 prompts
key knobs	LoRA rank 16–64 if used	β 0.05–0.1 (5 with length norm., Tülu 3)	G 8–16; ϵ 0.2 (0.28 upper); β 0 to 0.01; 4–8k tokens
compute	hours on 8 GPUs per 100k examples	hours	days; rollouts dominate

indicative ranges from Tülu 3, DAPO, Dr. GRPO and framework defaults; all are tuned per model and data.

Frameworks. TRL (Hugging Face), OpenRLHF, veRL (HybridFlow), slime, Axolotl, LLaMA-Factory; the first four run GRPO with vLLM or SGLang rollouts.

Summary: what each stage adds, and what it costs

Post-training is a sequence of cheap stages on top of one expensive one; each adds a behavior and a risk.

SFT	preference	RL (RLVR)	context	safety	tools
adds: format, instruction following, tool syntax	**adds:** taste, helpfulness, refusals	**adds:** reasoning, accuracy where checkable	**adds:** 128k windows	**adds:** refusal where it should	**adds:** calls, agents
risk: forgetting; copies the data's style	**risk:** length creep; over-optimization	**risk:** entropy collapse; reward hacking	**risk:** short-context regression	**risk:** over-refusal; a few tokens deep	**risk:** invented calls; credit assignment

Five things to remember

1. SFT is pretraining loss on the answer tokens; everything after it is a way to learn from a comparison or a reward instead of a demonstration. **2.** The reward model is a proxy and is optimized past its validity without a leash; verifiable rewards remove the proxy where a program can check the answer, which is why RL moved to math, code and tools first. **3.** GRPO is PPO without the critic, built for one reward per answer; the engineering problem is the rollout engine, not the gradient. **4.** Reasoning RL mostly sharpens what the base model can already sample; distillation moves it between models cheaply, and on-policy distillation is becoming the default middle stage. **5.** Context, safety and tool use are each a data problem with their own evals; each stage is checked by re-running every earlier suite.

Sources

Primary papers, checked against arXiv; † marks claims taken from release notes, blogs or commentary.

SFT and instruction data. Wei et al. 2021, FLAN, 2109.01652. Sanh et al. 2021, T0, 2110.08207. Ouyang et al. 2022, InstructGPT, 2203.02155. Wang et al. 2022, Self-Instruct, 2212.10560. Taori et al. 2023, Alpaca (GitHub). Zhou et al. 2023, LIMA, 2305.11206. Lambert et al. 2024, Tülu 3, 2411.15124. Hu et al. 2021, LoRA, 2106.09685. Dettmers et al. 2023, QLoRA, 2305.14314. Zelikman et al. 2022, STaR, 2203.14465.

Preference. Christiano et al. 2017, 1706.03741. Stiennon et al. 2020, 2009.01325. Bradley & Terry 1952. Gao et al. 2022, overoptimization, 2210.10760. Schulman et al. 2017, PPO, 1707.06347. Rafailov et al. 2023, DPO, 2305.18290. Azar et al. 2023, IPO, 2310.12036. Ethayarajh et al. 2024, KTO, 2402.01306. Meng et al. 2024, SimPO, 2405.14734. Hong et al. 2024, ORPO, 2403.07691. Ahmadian et al. 2024, RLOO, 2402.14740.

RL and reasoning. Shao et al. 2024, DeepSeekMath/GRPO, 2402.03300. DeepSeek-AI 2025, R1, 2501.12948. Yu et al. 2025, DAPO, 2503.14476. Liu et al. 2025, Dr. GRPO, 2503.20783. Wei et al. 2022, CoT, 2201.11903. Lightman et al. 2023, PRM800K, 2305.20050. Snell et al. 2024, test-time compute, 2408.03314. Muennighoff et al. 2025, s1, 2501.19393. Qwen3 2025, 2505.09388. Yue et al. 2025, pass@k, 2504.13837. OpenAI 2024, o1 (blog)†. Guha et al. 2025, OpenThoughts, 2506.04178. Sheng et al. 2024, HybridFlow/veRL, 2409.19256. IcePop: Ring-1T report†.

Context. Su et al. 2021, RoPE, 2104.09864. Chen et al. 2023, PI, 2306.15595. bloc97 2023, NTK-aware (Reddit)†. Peng et al. 2023, YaRN, 2309.00071. Xiong et al. 2023, ABF, 2309.16039. Ding et al. 2024, LongRoPE, 2402.13753. Grattafiori et al. 2024, Llama 3, 2407.21783. Liu et al. 2023, lost in the middle, 2307.03172. Hsieh et al. 2024, RULER, 2404.06654. Kamradt 2023, NIAH (GitHub). DeepSeek-V3 2024, 2412.19437.

Safety. Askell et al. 2021, HHH, 2112.00861. Bai et al. 2022, Constitutional AI, 2212.08073. Lee et al. 2023, RLAIF vs. RLHF, 2309.00267. Touvron et al. 2023, Llama 2, 2307.09288. Röttger et al. 2023, XSTest, 2308.01263. Zou et al. 2023, GCG, 2307.15043 (Table 2). Anil et al. 2024, many-shot jailbreaking (Anthropic)†. Mazeika et al. 2024, HarmBench, 2402.04249. Inan et al. 2023, Llama Guard, 2312.06674; Llama Guard 3 model card†. Guan et al. 2024, deliberative alignment, 2412.16339. Qi et al. 2023, fine-tuning compromises safety, 2310.03693. Qi et al. 2024, shallow safety alignment, 2406.05946. Li et al. 2024, WMDP/RMU, 2403.03218. Tamirisa et al. 2024, TAR, 2408.00761.

Tools and agents. Schick et al. 2023, Toolformer, 2302.04761. Qin et al. 2023, ToolLLM/ToolBench, 2307.16789. Liu et al. 2024, APIGen, 2406.18518. Patil et al. 2025, BFCL, ICML 2025 (PMLR 267; no arXiv). Yao et al. 2024, τ-bench, 2406.12045. Jimenez et al. 2023, SWE-bench, 2310.06770; SWE-bench Verified, OpenAI blog, Aug 2024†. Kimi Team 2025, Kimi K2, 2507.20534. NVIDIA NeMo Gym (GitHub)†.

Distillation and 2026 recipes. Agarwal et al. 2023, GKD, 2306.13649. Thinking Machines 2025, on-policy distillation (blog)†. Multi-teacher on-policy distillation 2026, 2606.30406. OLMo 3, AI2 Nov 2025 (release)†. DeepSeek-V4, GLM-5, MiMo-V2-Flash, Kimi K2.5, Nemotron 3: release notes and commentary (Interconnects, VESSL, Zylos, 2026)†.

Evaluation. Zhou et al. 2023, IFEval, 2311.07911. Zheng et al. 2023, MT-Bench, 2306.05685. Dubois et al. 2024, AlpacaEval 2 LC, 2404.04475. Li et al. 2024, Arena-Hard, 2406.11939. Jain et al. 2024, LiveCodeBench, 2403.07974. Rein et al. 2023, GPQA, 2311.12022. Hendrycks et al. 2020, MMLU, 2009.03300.

Curves labelled *illustrative* are sketches of a mechanism, not data; numbers labelled *derived* were computed from quoted ones. Hyperparameter ranges on the plan slide are indicative.